SDP



Module 5: MCP Servers

⏰ Copilot gets live context

"Skills package expertise. MCP servers connect that expertise to real systems."

M1 Instructions › M2 Plan Mode › M3 Prompts › M4 Skills › **M5 MCP** › M6 Agents

⏰ : Monday, 2:30 PM

R **Rafael** opens a feature ticket and realizes every answer still requires leaving the editor to check a live system.

> " Every time I open a ticket that touches live data, the first three answers I need all require leaving the editor. At some point that's not a tool limitation — it's a sprint tax. I'm not interested in paying it. "

Module 1
Document the standards
Structure and instructions created a reliable baseline.

Module 2
Plan the work
Plan mode made complex work reviewable.

Module 3
Save the best prompts
Repeated prompts became reusable entry points.

Module 4
Package expertise
Specialized workflows became teachable skills.

Module 5
Connect live systems
Now the goal is live databases, APIs, and services in the loop.

⚡ **MCP servers let Copilot query real tools and live resources without leaving the workflow.**

The Team, Before Module 5

⚠ WRONG ANSWERS ONLY

Jessica
The Eager Junior

"MCP is how Copilot connects to the internet. That's why it sometimes knows current stuff. It's already on."

Describing a completely different feature with total confidence.

David
The Seasoned Architect

"MCP is essentially SOAP with better branding and a younger demographic. I'll reserve judgment."

Technically wrong. Spiritually on brand.

Sarah
The Skeptical Senior

"MCP means Copilot can write directly to the database now. That's what 'live data access' means."

She's going to have follow-up questions about prod access.

Elena
The Quality Champion

"MCP servers run entirely in the cloud. There's nothing local to secure or audit."

The security concern is coming. It's just aimed at the wrong layer.

Marcus
The DevOps Developer

"MCP is just a fancy name for the GitHub extension. I already have that installed. Is there a different download?"

Confidently conflating two unrelated things.

Rafael
The Product Visionary

"MCP is already reading our Confluence docs, isn't it? I assumed that's what was happening."

It was not happening. He has been very wrong for weeks.

🎯 What You'll Build

3 exercises · ~45 minutes · rotating leads

Connect live data → upgrade the skill → run the full audit

5.1 — Marcus ⭐

Connect Live Lore Data

Configure an MCP server so Copilot can query live FanHub lore records directly in chat — no context-switching, no copy-paste.

You'll configure:

- `.vscode/mcp.json` for FanHub
- Live lore queries in Copilot Chat
- Validation that real data flows through

5.2 — Elena ⭐

Upgrade the Accuracy Skill

Extend `check-data-accuracy` to validate against the live database instead of the static universe file — and watch it catch what the old skill missed.

You'll see:

- Skills + MCP working in combination
- Errors the static skill would have missed
- Live DB records surfaced in skill output

5.3 — Sarah ⭐

Run the Lore Consistency Audit

Use both MCP servers together to catch DB-vs-universe mismatches and lore that exists in the database but never reaches the API.

You'll produce:

- A cross-source discrepancy report
- Orphaned lore records identified
- A reusable on-demand audit prompt

This module moves from static context to live operational context — and everything after this builds on that.

What MCP Servers Actually Add

A standard way to reach external systems

- MCP servers expose tools, resources, and prompts
- VS Code can talk to local or remote servers through a shared protocol
- The same pattern works across many MCP-compatible clients

Live context instead of copied context

- Query FanHub data without leaving chat
- Investigate duplicate records from live data instead of assumptions
- Validate runtime API responses against real lore expectations

Two common transport patterns

- `stdio` for local processes and tools
- `http` for hosted or remote services
- Choose transport based on security, locality, and deployment model

The unlock

Copilot stops depending on screenshots, pasted output, and stale docs. It can ask the source system directly.

"I've been pasting query results into chat for three sprints. This is the last time I do that."

— Sarah

🔎 Skills and MCP Work Better Together

Instructions

Define standards and expected behavior.

Skills

Package a domain workflow with examples, templates, and scripts.

MCP Servers

Bring live databases, APIs, repositories, and other systems into that workflow.

A strong pattern is: standards guide the work, skills package the workflow, MCP supplies live context.

"Oh — it's a stack. Instructions are config, skills are runbooks, MCP is the live system. I've been building this exact layering in bash for two years."

— Marcus

⚙ Core MCP Patterns in This Module

Local stdio servers

- Great for databases and local development tools
- Process runs locally and communicates over stdin/stdout
- Keeps access close to the workspace and avoids extra network hops

Runtime service wrappers

- Great for exposing a running API through an MCP-friendly wrapper
- Centralized checks can be reused across prompts, skills, and agents
- Useful when the source system is running outside the editor but still part of local development

Config

`.vscode/mcp.json` becomes the workspace entry point.

Discovery

VS Code discovers tools, resources, and prompts from each running server.

Invocation

Copilot can call the right tool when the task needs live system access.

❌ Before vs ✨ After

❌ Before

Manual lookup work keeps breaking flow

Schema checks, PR validation, and API inspection all require extra tabs, terminals, or tools.

Static documentation goes stale

The team can document expected behavior, but production reality still lives elsewhere.

Live validation is too slow to become habit

When every check is manual, the team does fewer checks and catches issues later.

✨ After

Live system questions stay in chat

Copilot can query databases, repositories, or APIs without requiring a context switch.

Earlier customizations become more useful

Skills and prompts can now operate on real data and live service output.

Validation gets faster and more credible

The team can confirm reality directly instead of depending on memory or copied snapshots.

🔨 Exercises — MCP

3 exercises · ~45 minutes · shared leadership

5.1 — Marcus ⭐

Connect live lore data

Configure an MCP server so Copilot can query live FanHub lore records directly in chat — no context-switching required.

You'll configure:

- `.vscode/mcp.json` for the FanHub database
- Live lore record queries in Copilot Chat
- Validation that real data flows through

5.2 — Elena ⭐

Upgrade check-data-accuracy Skill

Extend the `check-data-accuracy` skill to validate against the live database instead of the static universe file.

You'll see:

- Skills + MCP working in combination
- Errors the static skill would have missed
- Live DB records surfaced in skill output

5.3 — Sarah ⭐

Find the quote truncation bug

Use both MCP servers together to compare what the `quotes` table stores against what the API actually returns. There's a bug in the code — and it only shows up when you can read from both sources.

You'll discover:

- A truncation pattern in the API layer
- Which quotes are affected — and which aren't
- The difference between a data bug and a code bug

Each exercise rotates the lead person — everyone gets to drive at least once.

EXERCISE

5.1 Connect Live Lore Data to the Accuracy Workflow

Marcus ⭐ | 15 min

The `check-data-accuracy` skill validates entries against the universe file — but it can't catch errors that are ALREADY in the database. **This exercise connects Copilot to live FanHub lore data so the skill can cross-check real records, not just the universe file.**

STEPS

1. Create or update `.vscode/mcp.json` for the FanHub database connection
2. Start the SQLite MCP server and test lore-data queries from chat
3. Explore what records exist that could diverge from the universe file

SUCCESS CRITERIA

- [ ] Character and lore data is queryable from Copilot Chat
- [ ] Marcus can ask lore questions without leaving the editor
- [ ] The team can see what the real database contains before writing against it

> 💬 *Marcus: "Wait — it's showing me what's actually stored? Not what we think is stored? I was half-reading a Hacker News thread about data provenance — hold on, I need to re-read that now with this context."*

Example: .vscode/mcp.json — FanHub Database

```
{
  "servers": {
    "fanhub-db": {
      "command": "npx",
      "args": ["-y", "mcp-sqlite",
        "${workspaceFolder}/dotnet/Backend/fanhul
      ]
    }
  }
}
```

What this does

- Registers `fanhub-db` as a named MCP server in the workspace
- Runs `mcp-sqlite` via npx — no manual install required
- Points directly at the live FanHub database file on disk

NOW AVAILABLE IN CHAT

```
#fanhub-db what lore entries exist for Walter White?

#fanhub-db show me records where status or relationship fields look incomplete

#fanhub-db which entries might contradict the universe file?
```

Marcus:

Example: Copilot Chat — Lore Data Queries

Once `fanhub-db` is running in Agent mode, reference it directly to query live lore data:

LORE ENTRY LOOKUP

```
#mcp-fanhub-db Show me lore entries for Walter White and flag any that look incomplete or inconsistent.
```

CROSS-REFERENCE CHECK

```
#mcp-fanhub-db Show me character records where the canon status field is empty or missing — these might be accuracy check gaps.
```

DISCREPANCY HUNTING

```
#mcp-fanhub-db Compare lore entries in the database against what I'd expect from the universe file — flag anything suspicious.
```

💡 Each lore query stays in chat — no tab switch, no separate database tool, no copy-paste into context

EXERCISE

5.2 Upgrade check-data-accuracy with MCP

Elena ⭐ | 15 min

The `check-data-accuracy` skill from M3/M4 verifies lore entries against the universe file. **This exercise upgrades it to also query live FanHub data via MCP — so it catches discrepancies between what the universe file says and what's actually stored in the database.**

STEPS

1. Open `.github/skills/check-data-accuracy/SKILL.md` from M3/M4
2. Add a verification step that queries `#mcp-fanhub-db` to cross-check lore claims against live character records
3. Test: submit a lore entry that conflicts with a database record — does the skill catch it?

SUCCESS CRITERIA

- [ ] SKILL.md references `#mcp-fanhub-db` in its verification process
- [ ] Skill surfaces a DB vs. canonical discrepancy that file-only checks alone would miss
- [ ] Output still uses the Status / Issues / Recommendations format from M3/M4

💬 *Elena: "So before, the skill caught writing errors. Now it catches data entry errors too. Those are two different failure modes — and we were only checking one."*

Example: check-data-accuracy — MCP upgrade

```
---
name: check-data-accuracy
description: >
  Use this skill when asked to verify,
  validate,
  or fact-check a lore entry, character bio,
  seed data, or any show content.
  Triggers on: check lore, verify lore,
  fact-check, canon check, lore accuracy.
---

## Verification process
1. Read the content to check carefully
2. Read relevant sections of [show]-
universe.md
3. For each claim, find the corresponding fact
4. Query #mcp-fanhub-db to cross-check
  against live character records
5. Flag any DB-vs-universe discrepancies
6. Flag remaining discrepancies with correct
info
7. Note Unverifiable claims

## Resources
- docs/[show]-universe.md — canon source
- #mcp-fanhub-db — live records
```

What MCP adds

- **Before** — catches writing errors (wrong facts vs. universe file)
- **After** — also catches data entry errors (DB record contradicts universe file)
- **Two failure modes** — one skill now covers both

NEW OUTPUT SECTION

```
**DB Check** (if MCP available):
  - Live record matches → ✓ Consistent
  - Live record differs → flag discrepancy
  - No record found → note as missing
```

💬 Example: Invoking the Upgraded Skill

Add an incorrect lore fact manually, or ask Copilot "please add the lore item from #bad-lore-fact.json to the database"

```
can you validate the lore facts in the system?
```

What happens when you invoke it

1. Reads the selected lore entry and identifies every factual claim
2. Cross-checks each claim against `docs/[show]-universe.md`
3. **New with MCP** — also queries `#mcp-fanhub-db` to check what's actually stored in the live database
4. Returns **Status / Issues / Verified claims** — flagging both writing errors and DB discrepancies

Before vs. After

~~Static only~~ — catches writing errors vs. universe file

MCP-upgraded — also catches DB records that contradict canon

💬 *Elena: "The static check told us what we expected. The MCP check tells us what is stored. I'd been treating those as the same question."*

EXERCISE

5.3 Find the Quote Truncation Bug

Sarah ⭐ | ⏰ 20 min

The quotes feature is live. The database looks fine. But when you run `check-data-accuracy` with *both* MCP servers and compare what's stored against what the API returns — **something doesn't match. Long quotes come back shorter. There's a bug in the code truncating `quote_text` to 50 characters — but it's invisible until you can read from both sources.**

📋 STEPS

1. Use `#mcp-fanhub-db` to query the `quotes` table — inspect some records, look at the `quote_text` field
2. Run `/check-data-accuracy` on a few quotes — does anything look unexpectedly short?
3. Add `fanhub-api` to `.vscode/mcp.json` if not already done
4. Use `#mcp-fanhub-api` to fetch the same quotes — compare `quote_text` in the DB vs. the API response
5. Identify the pattern: which quotes are affected? Can you find the 50-char limit in the code?

✅ SUCCESS CRITERIA

- [ ] At least one quote where `quote_text` in DB ≠ API response
- [ ] Truncation pattern documented — quotes over 50 chars are silently cut
- [ ] Short quotes (≤ 50 chars) pass through correctly — the pattern is consistent
- [ ] Sarah can say: "The data is correct. The API is wrong. There are two different facts."

💬 *Sarah: "The DB is fine. The API is lying. The skill only caught it because it could read from both — without both servers open, this bug is invisible."*

Example: .vscode/mcp.json — Both Servers

```
{
  "servers": {
    "fanhub-db": {
      "command": "npx",
      "args": ["-y", "mcp-sqlite",
        "${workspaceFolder}/dotnet/Backend/fanhul
      ]
    },
    "fanhub-api": {
      "type": "stdio",
      "command": "node",
      "args": ["./mcp-servers/fanhub-api-
      server.js"],
      "env": {
        "FANHUB_API_URL":
        "http://localhost:5265"
      }
    }
  }
}
```

fanhub-db

SQLite MCP server via `mcp-sqlite`

Exposes the database directly — schemas, records, relationships

`#mcp-fanhub-db ...`

fanhub-api

Custom stdio server wrapping the *running* FanHub API at `localhost:5265`

Exposes what the API actually returns to clients — not what's stored

`#mcp-fanhub-api ...`

Together they cover two failure modes: wrong data stored and correct data not surfaced

Example: check-data-accuracy — API upgrade

```
---
name: check-data-accuracy
description: >
  Use this skill when asked to verify,
  validate,
  or fact-check a lore entry, character bio,
  seed data, or any show content.
---

## Verification process
1. Read the content to check carefully
2. Read relevant sections of [show]-
universe.md
3. For each claim, find the corresponding fact
4. Query #mcp-fanhub-db to cross-check
  against live character records
5. Flag any DB-vs-universe discrepancies
6. Query #mcp-fanhub-api to verify the entry
  is actually returned by the running API
7. Flag any DB records missing from API
responses
8. Flag remaining discrepancies with correct
info
9. Note Unverifiable claims

## Resources
- docs/[show]-universe.md — canon source
```

What the API check adds

- **DB check** — catches data stored wrong (canon violation)
- **API check** — catches correct data silently dropped or truncated before the client sees it
- **Three failure classes** — one skill now covers all of them

NEW OUTPUT SECTION

```
**API Check** (if #mcp-fanhub-api available):
  - Entry returned by API → ✓ Surfaced
  - Entry missing from API → flag as API gap
  - Entry truncated → flag missing fields
```

📋 Example: The Skill Covers All Three

SKILL INVOCATION — UNCHANGED

```
can you validate the quotes in the system?

// 1. checks universe file (writing errors)
// 2. queries #mcp-fanhub-db (DB-vs-universe errors)
// 3. queries #mcp-fanhub-api (DB-vs-API gaps)
```

Same invocation. Both MCP servers are reached automatically when available.

📊 THREE FAILURE CLASSES — NOW ALL COVERED

`DB error + API exposes it`

`DB correct + API drops it`

`DB error + API hides it`

Each class gets a different priority. The skill now tells you which one you're looking at.

Before vs. After

- ~~Static only~~ — catches writing errors vs. universe file
- + DB check — also catches data stored wrong
- + API check — also catches correct data never surfaced

Elena:

📋 Example: Audit Output — Truncation Bug Found

DB Check (via #mcp-fanhub-db):

Status: Consistent with universe file

Sample records (quotes table):

- quote-001: "Say my name."
- quote-007: "I am the one who knocks. I am the danger. A danger to your operation."
- quote-012: "Chemistry is the study of change. That's all of life."

All DB records: full text intact ✓

API Check (via #mcp-fanhub-api):

Status: Discrepancy Found

Issues:

- quote-001: "Say my name." — ✓ matches
- quote-007: "I am the one who knocks. I am the dan"
  ↳ DB: 70 chars | API: 50 chars — truncated
- quote-012: "Chemistry is the study of change. That's"
  ↳ DB: 55 chars | API: 50 chars — truncated

Pattern:

All quotes over 50 chars are silently cut. Shorter quotes pass through unchanged.

💬 *Sarah: "The data is correct. The code is wrong. I can prove it — forward. I can't find the issue, but the API response looks fine now."*
Elena: "50-character hard limit. That's not an accident, that's a `LEFT(quote_text, 50)` somewhere. Find it."

Example: mcp-servers/fanhub-api-server.js

```
// stdio MCP server — wraps the running FanHub
API
const API_BASE_URL =
process.env.FANHUB_API_URL ||
"http://localhost:5265";

// Handshake
if (method === "initialize") {
  send(id, { protocolVersion: "2024-11-05",
    serverInfo: { name: "fanhub-api" } });
}

// Three tools exposed to Copilot
if (method === "tools/list") {
  { name: "get_lore",
   description: "Fetch all lore entries from
   the API" },
  { name: "get_lore_by_id",
   inputSchema: { id: { type: "string" } } },
  { name: "create_lore",
   inputSchema: { showId, category, title,
   description } }
}
```

How it works

- Pure stdio — reads JSON-RPC from stdin, writes responses to stdout. No port, no daemon.
- `FANHUB_API_URL` env var points it at the running API — swap the URL for any environment
- VS Code starts it automatically when the workspace opens via `.vscode/mcp.json`

Three tools

- `get_lore` — list all entries; used to compare count vs. DB
- `get_lore_by_id` — spot-check a specific record the DB audit flagged
- `create_lore` — create a corrected entry without leaving the workflow

The audit can fix, not just find

The `create_lore` tool means Copilot can propose and apply corrections directly — the skill doesn't just report what's wrong, it can write the fix.

✅ Module Checklist

Connect

- Add the right MCP server configuration for the scenario
- Pick the transport that fits the system and security model
- Confirm the server starts and exposes useful capabilities

Apply

- Use live context to improve a real team workflow
- Combine MCP with prompts, standards, or skills where it helps
- Keep the integration understandable and easy to reuse

Validate

- Measure whether manual effort drops
- Check that outputs are grounded in live data
- Refine the workflow until the connection is genuinely worth keeping

A good MCP integration reduces switching, grounds the workflow in live systems, and increases confidence.

Keep the Foundation Evergreen

Before Module 6, document the live context you've wired in.

Update `ARCHITECTURE.md` and `.github/copilot-instructions.md` so the agent knows what data sources and MCP servers are available.

You already built the tool. Run it.

`/refresh-docs`

The **`refresh-docs.prompt.md`** you built in Module 3 updates **`ARCHITECTURE.md`** and **`copilot-instructions.md`** with the new MCP servers and live data sources — so Module 6 agents know they exist and can use them.

BUILD THE HABIT

Run `/refresh-docs` after every meaningful change.

Each PR that updates docs keeps the foundation relevant — so Copilot always has accurate context to work from.

SDP

Module 5 Is Locked In

🤖 Next Up: Module 6 — Custom Agents

You Connected Live Data

FanHub character data is now queryable directly from Copilot chat

You Upgraded the Skill

The character-change workflow now operates on real records, not guesswork

Now Build the Agent

Module 6 combines prompts, skills, and live data into one orchestrated workflow

MCP servers let Copilot see the real world. **Custom agents will let it act on all of it at once.**

Reflect & Discuss

STORY MOMENTS

Marcus's grounding moment: "it's showing me what's *actually* stored? Not what we think is stored?" — assumptions met reality

Two failure classes surfaced in one session: Sarah found a DB error the API masked; Elena found records silently dropped between layers

Sarah ran checks at 2:30pm she'd planned to do manually at 5pm — the tool got faster than her instinct to postpone

ONE BIG QUESTION

What's a question you've answered this week that required switching to another tool — a database, a dashboard, a doc? What would it mean to answer that without leaving the workflow?

The Team, After Module 5

✓ AFTER THE MODULE

Jessica

The Eager Junior

"It's not the internet connection — it's live data from the actual database. I kept waiting for it to be wrong. Three more runs. It was not the internet. It was worse: it was correct."

Testing the tool to find the excuse. Didn't find it.

David

The Seasoned Architect

"It is not SOAP. The correct term is 'grounding' — anchoring inference in verified external state rather than learned approximation. I retract the SOAP comparison. Partially."

Corrected himself before anyone else could.

Sarah

The Skeptical Senior

"It doesn't write to the database — it reads from it. Three lore errors. Two API gaps. One regression before the PR. I was going to run those checks manually at 5pm. It's 2:30."

Wrong about the direction. Right that it changes everything.

Elena

The Quality Champion

"They don't run in the cloud — they run locally and that surface needs to be audited. DB error confirmed. API gap confirmed. Seventeen edge cases queued. I am professionally obligated to keep checking."

Found the local surface. Already auditing it.

Marcus

The DevOps Developer

"It is not the GitHub extension — I have that and it does not do this. I was reading an MCP internals thread while it ran, looked up, and it had already found the thing. Adding it to CI. Two things."

Two parallel trains. Already shipping before he finished reading.

Rafael

The Product Visionary

"It was not reading Confluence. It was catching data errors I didn't know existed. The three failure classes map to two epics and a Q3 OKR. We're shipping this."

Tuned out the gap. Made the call anyway.